

Graph coloring

Решение

Отжиг

Шедулинг

- температура убывает экспоненциально, множитель 0.9999
- вероятность перехода обычная, $P = \exp\left(-\frac{\Delta_{\text{score}}}{T}\right)$

Состояние

Перестановка вершин, задающая порядок, в котором мы жадно красим граф.

Старт

id

Переходы

Выбираем случайную стартовую вершину и начнём из неё случайное блуждание по графу по каким-то весам с каким-то распределением длин, при этом запрещая переходить в вершину из которой пришли.

Далее переставляем вершины в порядке нашего случайного пути.

Кост-функция

Число цветов в жадной раскраске.

Первая итерация

Экспоненциальный шедулер с множителем 0.9999, распределение длин путей экспоненциальное, средняя длина $\sqrt{n}$.

Instance	Score	CPU time, s	Points
gc_50_3	7	0.29	3
gc_70_7	18	0.89	3
gc_100_5	17	1.29	3
gc_250_9	83	13.46	3
gc_500_1	17	6.95	3
gc_1000_5	120	203.51	3

Итого: 18 / 30.

Вторая итерация

Попробуем переставлять вершины из нашего случайного пути по циклу, а не в порядке пути.

Instance	Score	CPU time, s	Points
gc_50_3	6	0.14	5
gc_70_7	18	0.50	3
gc_100_5	17	0.73	3
gc_250_9	85	7.34	3
gc_500_1	17	3.80	3
gc_1000_5	120	93.23	3

Итого: 20 / 30.

Третья итерация

Вышло классно, давайте поменяем множитель шедулера с 0.9999 на 0.99999.

Instance	Score	CPU time, s	Points
gc_50_3	6	1.43	5
gc_70_7	17	4.97	5
gc_100_5	16	7.29	5
gc_250_9	81	73.06	3
gc_500_1	17	36.58	3

Итого на пяти тестах: 21 / 25, 1000 пока не будем считать, жирновато.

Четвёртая итерация

Давайте попробуем поменять распределение длин. Например, на экспоненциальное с матожиданием в $\log_2 n$.

Instance	Score	CPU time, s	Points
gc_50_3	6	1.31	5
gc_70_7	17	4.35	5
gc_100_5	16	6.10	5
gc_250_9	79	54.36	3
gc_500_1	17	27.20	3

Итого на пяти тестах: 21 / 25. Побыстрее и где-то даже получше.

Пятая итерация

Давайте попробуем нормальное с матожиданием в $\sqrt{n}$.

Instance	Score	CPU time, s	Points
gc_50_3	6	1.51	5
gc_70_7	17	5.07	5
gc_100_5	16	7.29	5
gc_250_9	82	70.67	3
gc_500_1	17	34.82	3

Итого на пяти тестах: 21 / 25. Медленнее и без понятного профита, откатим.

Шестая итерация

Вернём экспоненциальное распределение длин с матожиданием в $\log_2 n$ и будем семплировать случайные пути только из вершин, на которых жадник начал новый цвет.

Instance	Score	CPU time, s	Points
gc_50_3	6	1.35	5
gc_70_7	17	4.57	5
gc_100_5	16	6.46	5
gc_250_9	81	72.12	3
gc_500_1	17	29.36	3

Итого на пяти тестах: 21 / 25. Медленнее без понятного профита, откатим.

Седьмая итерация

(тут я заметил, что компилировал всё в дебажной сборке и поэтому всё так медленно работало)

Сиды: 664222986, 906064618, 2632799159.

Instance	Score 1	CPU 1, s	Score 2	CPU 2, s	Score 3	CPU 3, s
gc_50_3	7	0.19	6	0.16	6	0.16
gc_70_7	17	0.54	18	0.50	18	0.50
gc_100_5	17	0.76	18	0.72	17	0.71
gc_250_9	84	6.77	85	6.43	87	6.42
gc_500_1	17	3.73	18	3.39	17	3.35
gc_1000_5	119	78.46	120	76.77	119	76.94

Итого: 20 / 30, 20 / 30, 20 / 30.

Восьмая итерация

У чистого счётчика цветов сильно не хватает градиента, он редко меняется. Добавим информации: давайте штрафовать малое количество оставшихся цветов у вершины в процессе раскраски. Как штрафовать? Заметим, что оставшееся количество 1 сильно хуже оставшегося количества 2 и так далее. Поэтому давайте штрафовать экспоненциально с множителем $1/2$. И чтобы сохранить выгодность улучшения, давайте разделим этот скор на число вершин.

Сиды: 152482821, 2828801481, 1907818124.

Instance	Score 1	Score 2	Score 3	Median CPU, s	Points
gc_50_3	6	7	6	0.44	5
gc_70_7	17	18	18	1.66	3
gc_100_5	17	17	17	2.58	3
gc_250_9	84	84	84	26.63	3
gc_500_1	17	16	16	12.93	5
gc_1000_5	118	120	118	236.40	3

Итого по медиане: 22 / 30.